AIMERZ.ai
Aim.Act.Achieve

NOTES

Java Object-Oriented Programming (OOP) - I

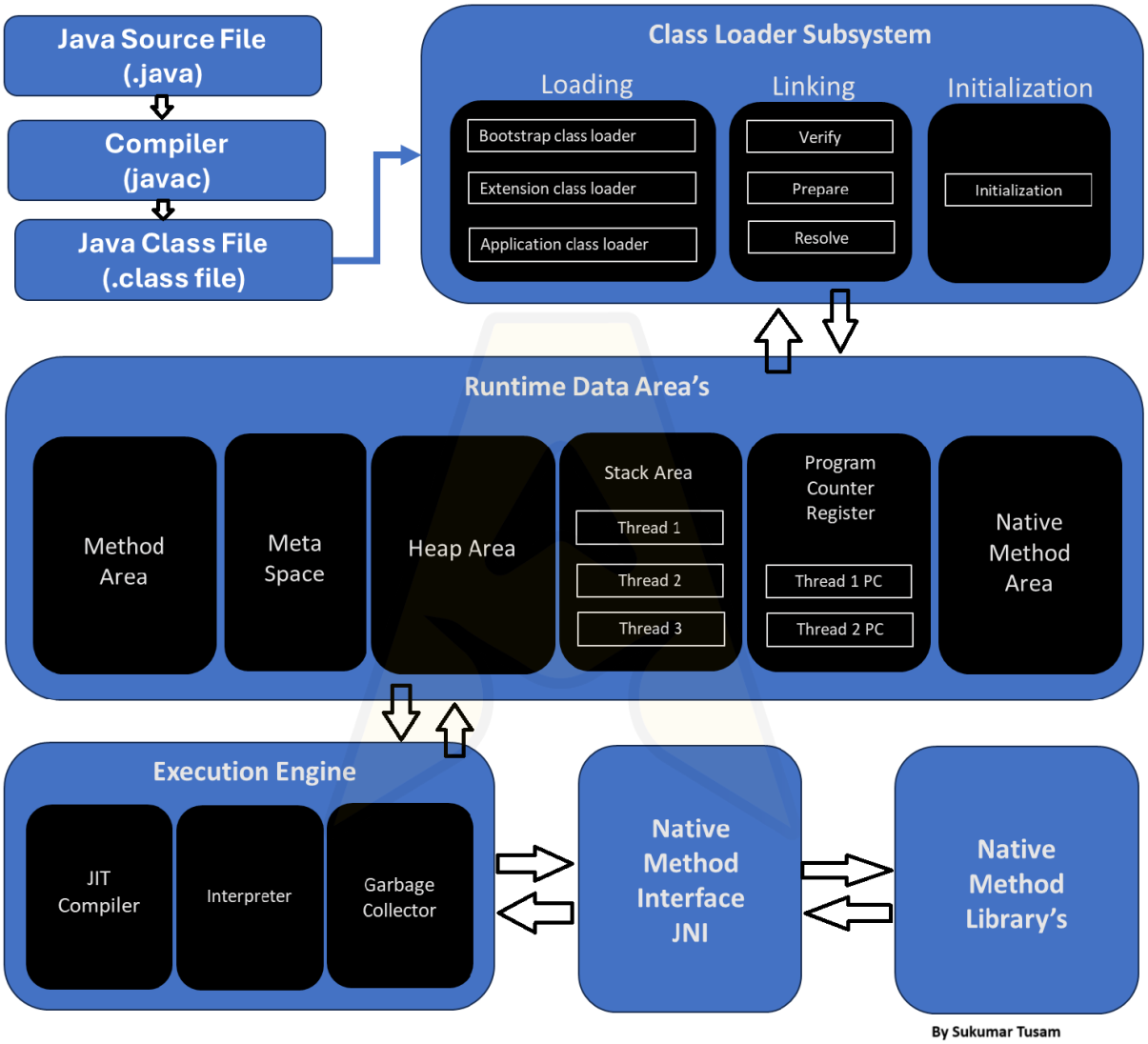
JVM Memory Areas

Static Members

JVM Memory Areas

When we execute a .java file, this process involves multiple steps, and different parts of memory are used at various stages.

So what happens after executing a .java file step by step and where each component and in which memory (Heap, Stack, etc.) comes into play.



Process After Executing a .java File

Compilation:

When we run a .java file, the **Java compiler (javac)** first converts the human-readable Java code into **bytecode** (.class file), which is understood by the Java Virtual Machine (JVM).

Loading and Execution:

Once we have the .class file (bytecode), the **JVM** loads and executes it.

1. Class Loader:

The JVM's **Class Loader** reads the .class file and loads the class into the **Method Area**.

2. Method Area:

It stores the class structure like method code, static variables, etc. It knows which methods and static variables are associated with the class.

3. Execution of main():

The execution will start at the main() method or whichever method we call to start.

The **PC Register (Program Counter)** keeps track of which part of the method is currently being executed.

4. Method Call & Stack:

When a method is called like main() or any other method, its local variables and current execution state are pushed/stored onto the **Stack**.

5. Stack:

This stores **local variables** (variables that are declared inside a method) and **method calls**. (when a method is 0

For example, in main(), if we declare int x = 10;, the value of x is stored in the **Stack** for that method call.

6. Object Creation & Heap:

Whenever we create an object (e.g: Car myCar = new Car()), the object is created in the **Heap**.

7. Heap:

This stores objects and their **instance variables** (e.g: model, year of the Car class).

If myCar has instance variables model and year, those are stored in the **Heap**, and the reference to myCar is stored in the **Stack**.

8. Native Method Invocation & Native Method Stack:

If our code calls a **native method** e.g: a method written in C/C++, the JVM will execute it using the **Native Method Stack**. This part is used for methods that are not written in Java but are essential for low-level operations.

9. Garbage Collection:

Java's **Garbage Collector** automatically frees up memory in the **Heap** by removing objects that are no longer referenced anywhere in our code.

Static Members in Java

What Are Static Members?

In Java, **static members** (variables or methods) belong to the class, not to individual objects. This means:

- **Static Variables**: Shared by all objects of the class. There's only one copy for the entire class.
- **Static Methods**: Can be used directly without creating an object of the class.

Key Features of Static Members

Shared: All instances share the same static variable. Changing it affects all objects.

Accessed Directly: Static members can be accessed using the class name without creating an instance.

Real-World Analogy: Announcement Board

Imagine a **school's announcement board**. All students see the same announcements, regardless of which class they belong to. If the announcement changes, every student sees the updated information instantly.

In Java:

The **announcement** is like a static variable – shared by all students (instances).

The **school board** represents the class containing the announcement (static variable).

Example 1: Static Variable for a Library's Book Count

```java
class Library {
    // Static variable to keep track of total books in the library
    static int totalBooks = 0;

    // Instance variable for each book's title
    String title;

    // Constructor that increases the totalBooks count each time a new book
is added
    public Library(String bookTitle) {
        title = bookTitle;
        totalBooks++;
    }
}

public class LibraryTest {
    public static void main(String[] args) {
        // Creating book objects
        Library book1 = new Library("Java Programming");
        Library book2 = new Library("Python Essentials");

        // Accessing the static variable totalBooks using the class name
        System.out.println("Total Books in Library: " + Library.totalBooks);
    }
}
```

Output is 2

```java
Total Books in Library: 2
```

Explanation

totalBooks is a static variable, shared across all instances of Library.

Every time a new Library object is created, totalBooks increases.

Static Method in Java

A **static method** in Java is a method that belongs to the class rather than any specific instance (object) of that class. This means we can call a static method without creating an instance of the class.

Key Points about Static Methods

Class-Level Method: Static methods are called on the class itself, not on instances.

No Object Required: You don't need to create an object to use a static method.

Access to Static Variables: Static methods can access static variables and other static methods directly.

Cannot Access Instance Variables: They cannot access instance variables (non-static variables) directly; you need an object to do that.

Real-World Analogy: Lectures

"Imagine a **school** where teachers give lectures. The lectures (static methods) are available to all students (class instances) at any time. Students can attend the lectures without needing to create a separate classroom (object). However, to ask a specific question about their grades (instance variable), they need to meet their respective teachers in their classrooms (objects)."

Example 1: Basic Static Method

Let's look at a simple example of a static method that calculates the square of a number.

```java
class MathUtils {
    // Static method to calculate square
    public static int square(int number) {
        return number * number; // Returns the square of the number
    }
}

public class MathTest {
    public static void main(String[] args) {
        // Calling the static method without creating an object
        int result = MathUtils.square(5);
        System.out.println("Square of 5 is: " + result);
    }
}
```

Output

```java
Square of 5 is: 25
```

Explanation

The square method is a static method defined in the MathUtils class.

It calculates the square of the number passed to it.

You can call MathUtils.square(5) directly without creating an instance of MathUtils.

Example 2: Static Method with Utility Functions

Static methods are often used for utility functions that do not require object state. Let's create a utility class for Arithmetic operations.

```java
class NumberUtils {
    // Static method to check for even number or not
    public static boolean CheckEven(int num) {
        return num % 2 == 0; // Checking the given number
    }

    // Static method to print all even numbers in the Loop
    public static void printEven(int num) {
        for (int i = 0; i < num; i++) {
            if (i % 2 == 0) {
                System.out.println("Is Even:: "+i);
            } else {
                System.out.println("Not Even:: "+i);
            }
        }
    }
}

public class StringTest {
    public static void main(String[] args) {
        // Using static method to check a number
        boolean checkNum = NumberUtils.CheckEven(21);
        System.out.println("This Number is Even:: " + checkNum);

        // using static method to print even numbers
        NumberUtils.printEven(10);

    }
}
```

Output

```java
This Number is Even:: false
Is Even:: 0
Not Even:: 1
Is Even:: 2
Not Even:: 3
Is Even:: 4
Not Even:: 5
Is Even:: 6
Not Even:: 7
Is Even:: 8
Not Even:: 9
```

Explanation

The CheckEven method is a static method in the NumberUtils class that takes a number and returns true or false.

It can be called directly without creating an instance/object of NumberUtils.

And the in the same way the static method printEven can also be called without creating the object of NumberUtils class and print all even and odd numbers

Example 3: Static Method in a Banking Application

Let's take a bigger example where a static method calculates the interest for a bank account.

```Java
class Bank {
    static double interestRate = 0.05; // Static variable for interest rate

    // Static method to calculate interest
    public static double calculateInterest(double principal, int years) {
        return principal * interestRate * years; // Calculates simple
interest
    }
}

public class BankTest {
    public static void main(String[] args) {
        double principal = 1000; // Principal amount
        int years = 3; // Number of years

        // Calling the static method to calculate interest
        double interest = Bank.calculateInterest(principal, years);
        System.out.println("Interest for " + years + " years: Rs: " +
interest);
    }
}
```

Explanation

The calculateInterest method is a static method that calculates simple interest based on the principal amount and number of years.

It uses a static variable interestRate.

This method can be called directly using Bank.calculateInterest() without creating a Bank object.

Output:

```
Java
Interest for 3 years: Rs: 150.0
```

Final Brief on Static Methods

Static Methods: Methods that belong to the class rather than an instance. They can be called without creating an object of the class.

Use Cases: Commonly used for utility or helper functions, calculations, and configurations that do not depend on object states.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan